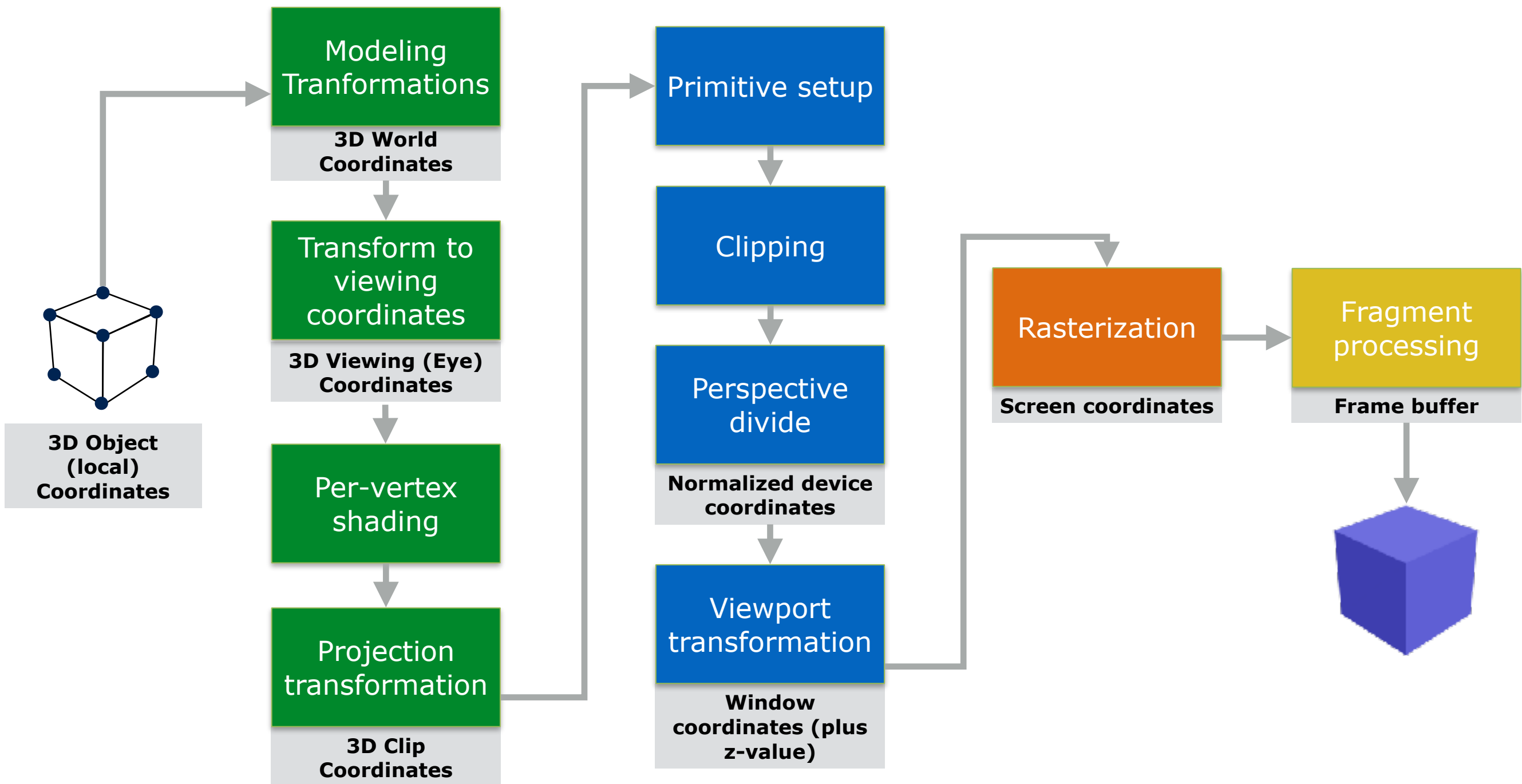


Fragment processing

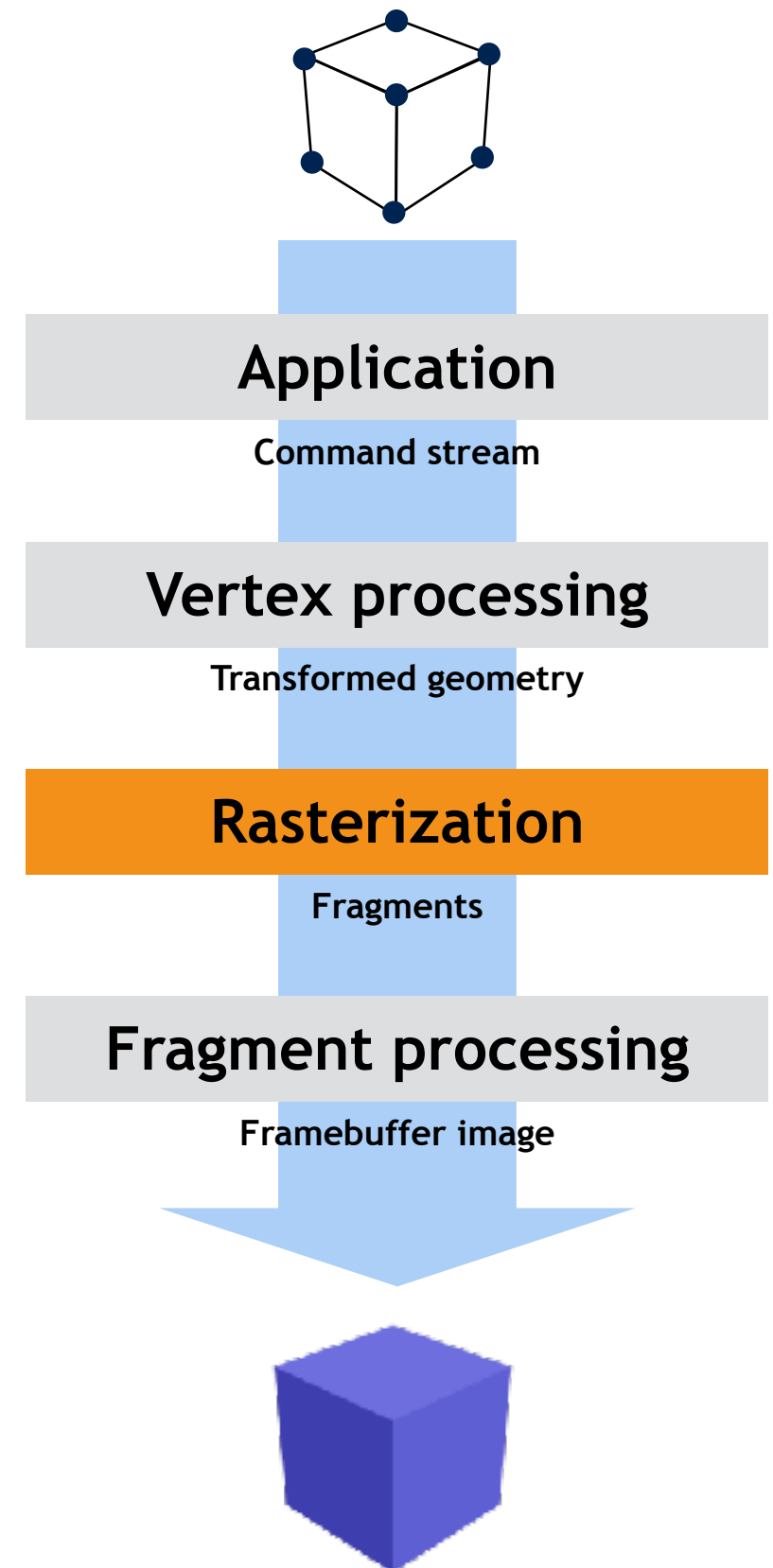
Coordinate systems



Real-time rendering pipeline

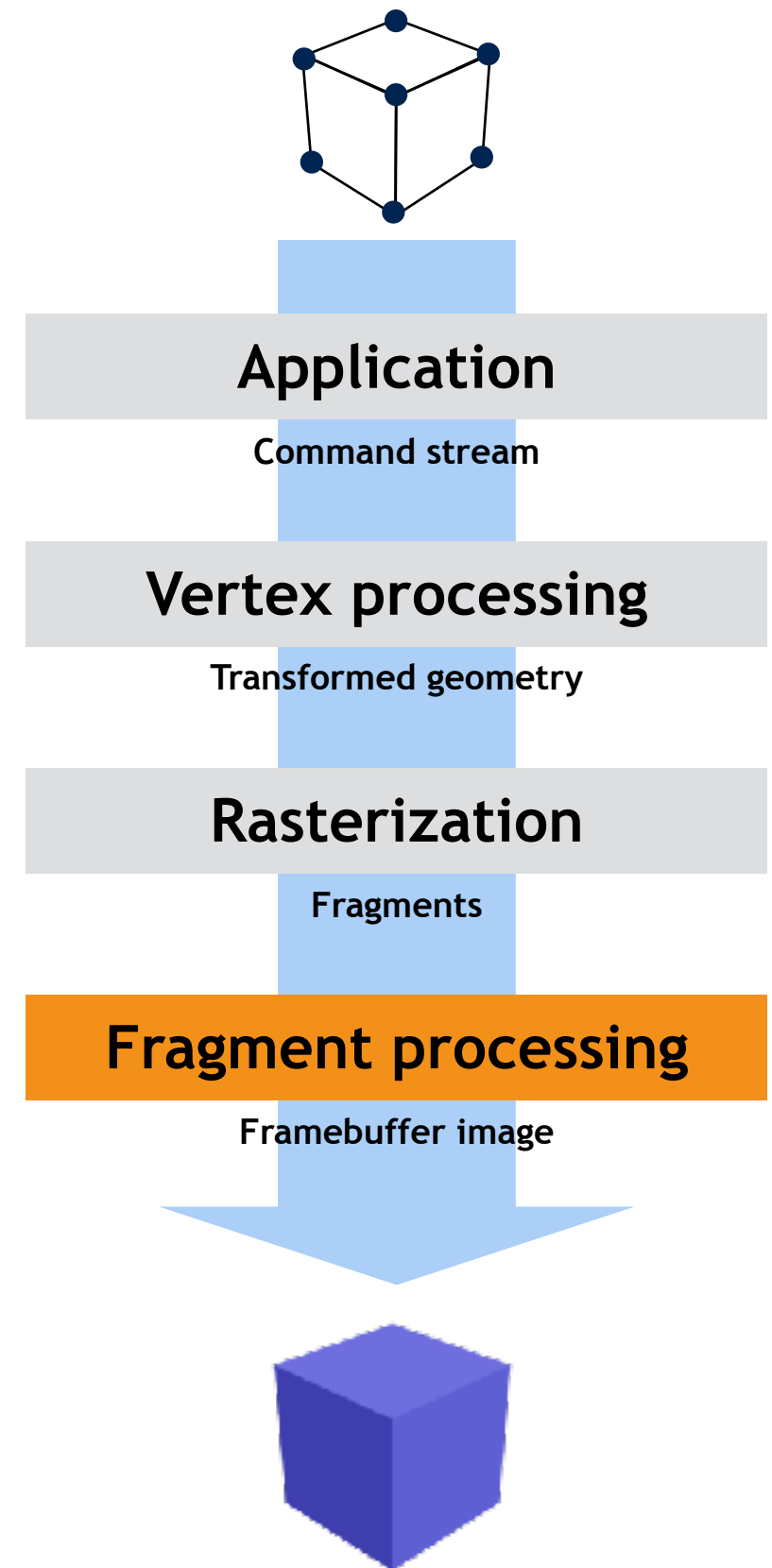
- **Rasterization**

- **enumerates** the pixels that are covered by the primitive
- **interpolates** values, called attributes, across the primitive
- The output of the rasterizer stage is a set of **fragments**, one for each pixel covered by the primitive
- Fragment data
 - raster position
 - depth (z-value)
 - interpolated attributes (color, texture coordinates, etc.)
 - stencil
 - alpha

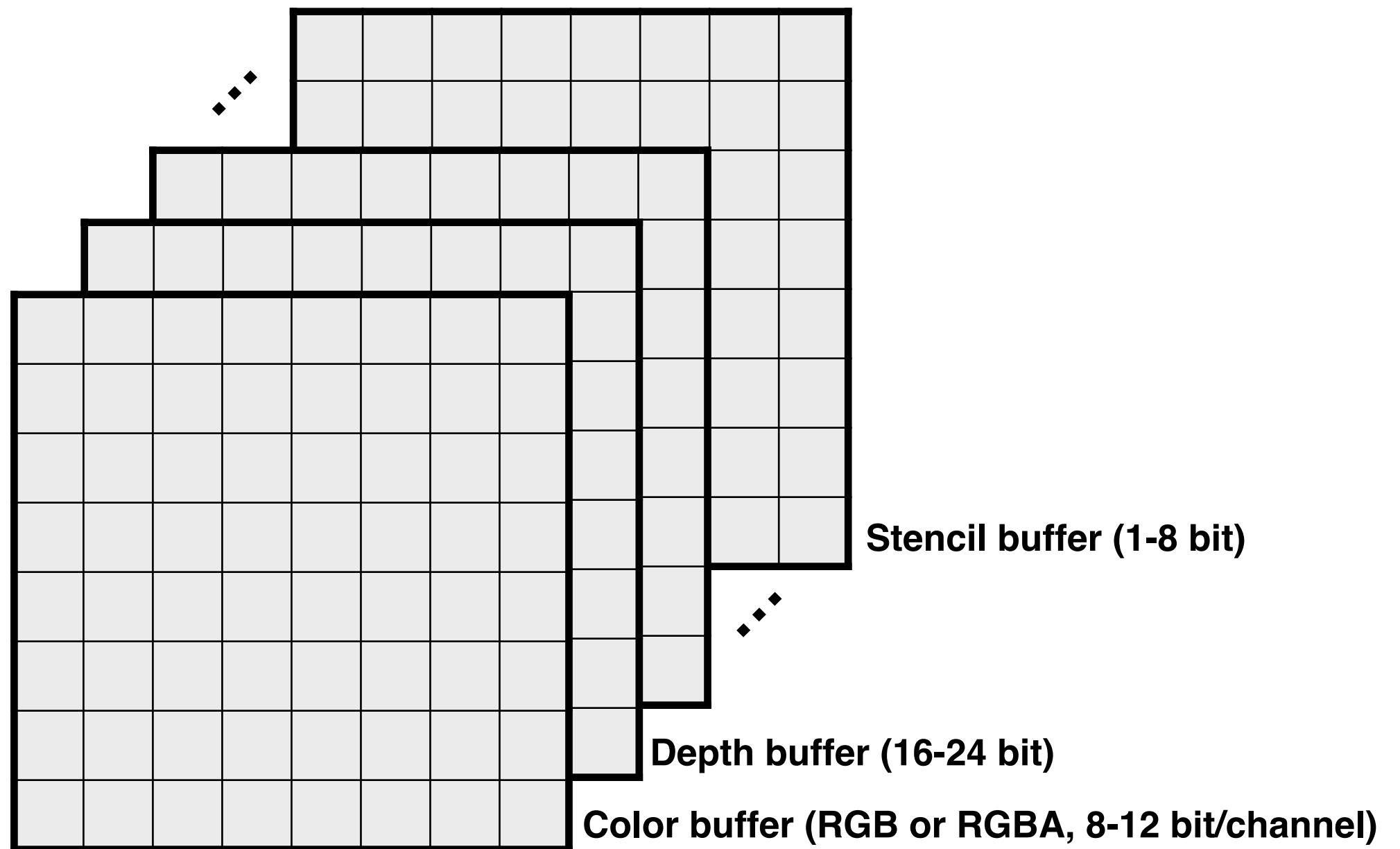


Fragment processing

- Process each fragment from the rasterization process into a set of colors and a single depth value
- **Scissor test** - discard fragments outside of a certain rectangular portion of the screen
- **Stencil test** - test the fragment's stencil value against the value in the current stencil buffer; if the test fails, the fragment is culled
- **Depth test** - test the depth of the current fragment with the existing depth; if the test passes update the depth buffer
- **Blending** - combine the fragment's color with the existing color in the frame buffer



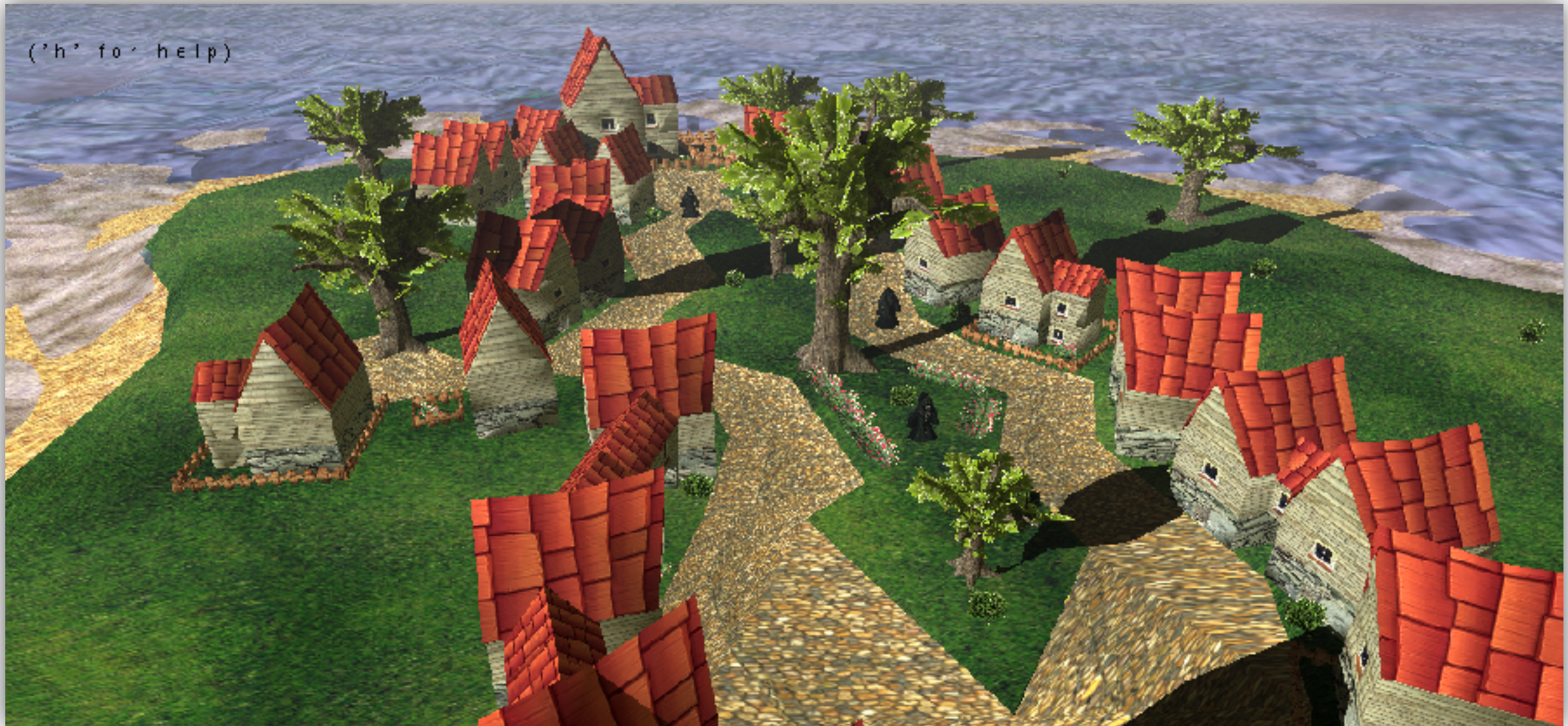
Framebuffer



Illumination

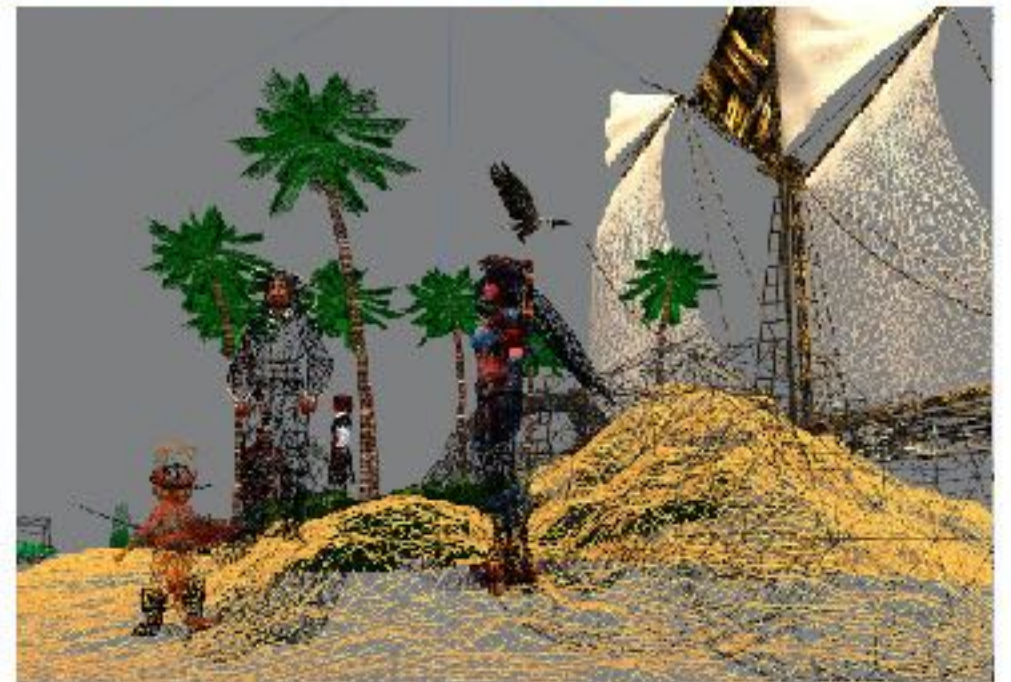
- Compute light intensity on each object point (fragment)
- **Reflection model**
 - **Local** – direct illumination
 - **Global** – direct illumination + multiple reflections
- Influenced by
 - **Light model**
 - Ambient illumination
 - Diffusion illumination
 - Specular illumination
 - **Materials**

Local vs Global illumination



[GPS Project – Simina Alexandru]

Local vs Global illumination



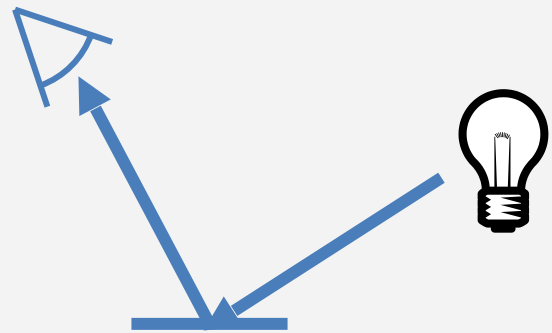
[GPS Project – Somfelean Iulia]

Local vs Global illumination

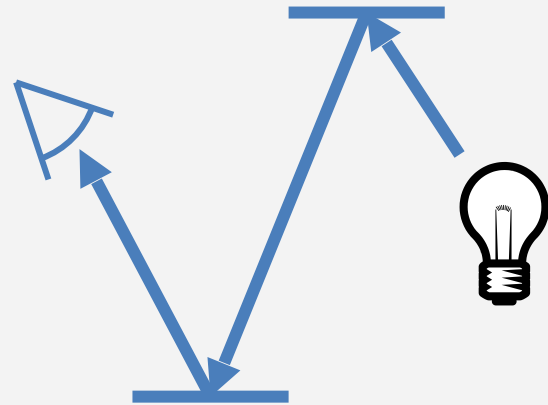


[<http://simonmajar.com/storage/originals/e6/d3/screenshot0000.jpg>]

Global illumination

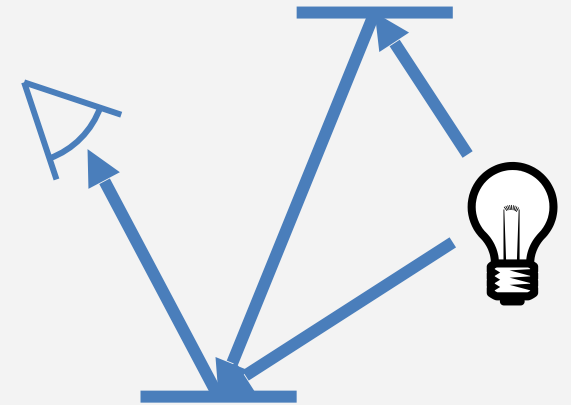


Direct illumination

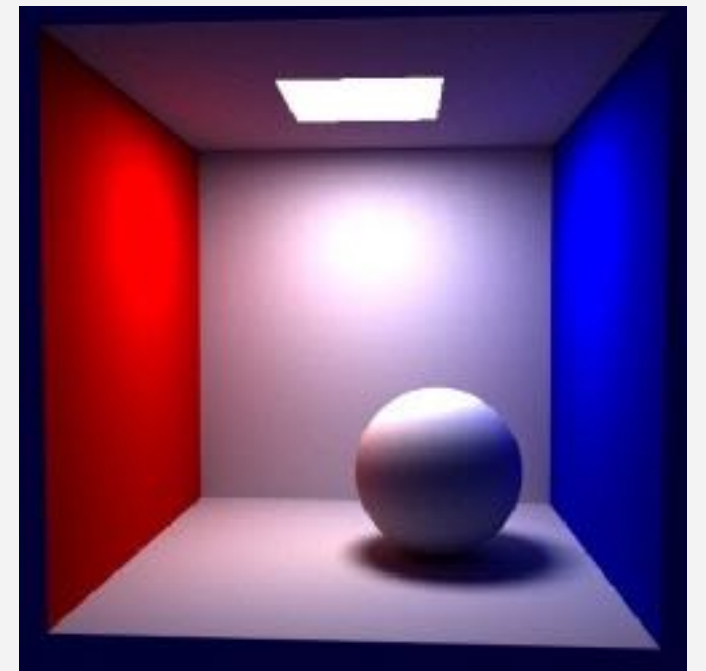
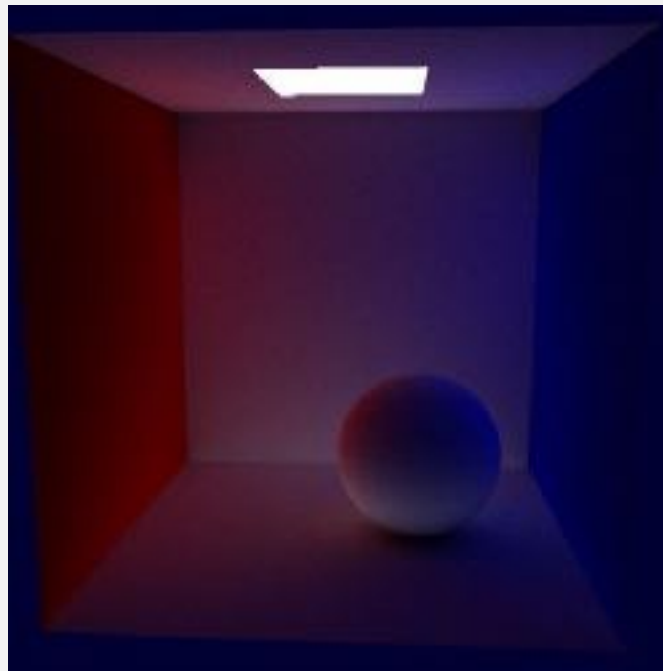
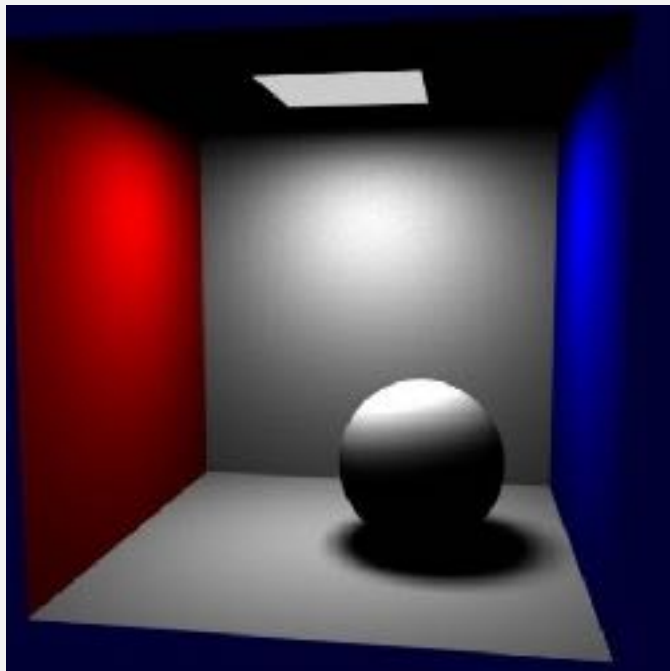


Indirect illumination

=



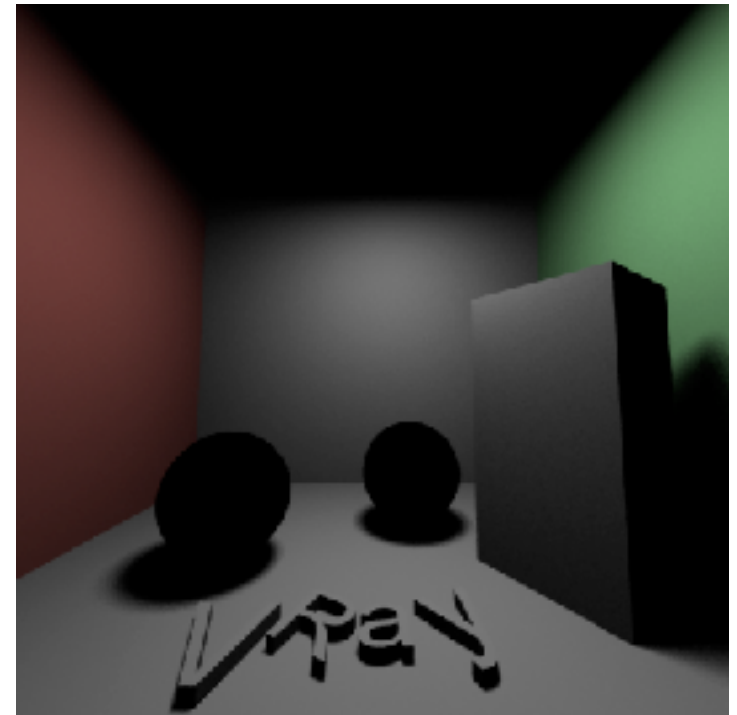
Total illumination



Illumination models

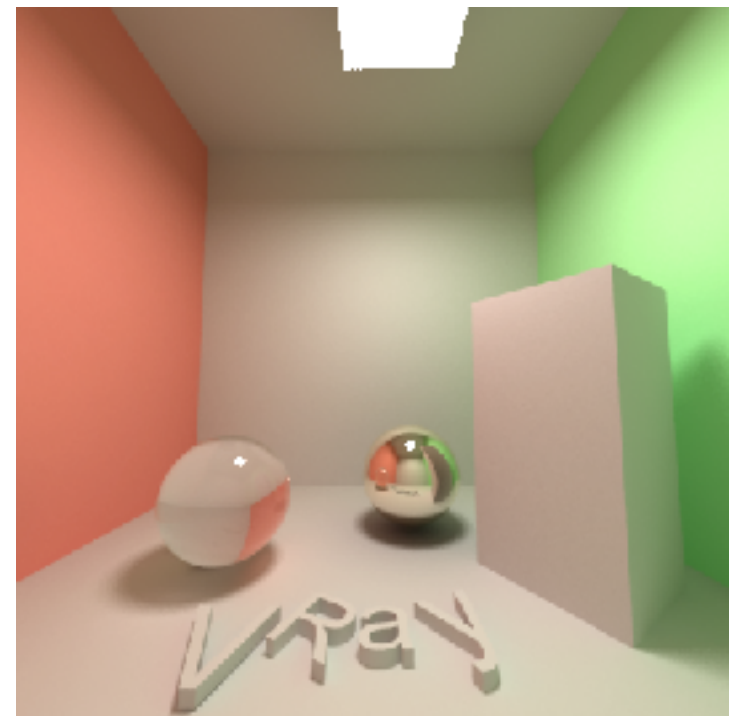
- **Local models**

- light from direct sources
- advantages: fast rendering
- disadvantages: low photorealism



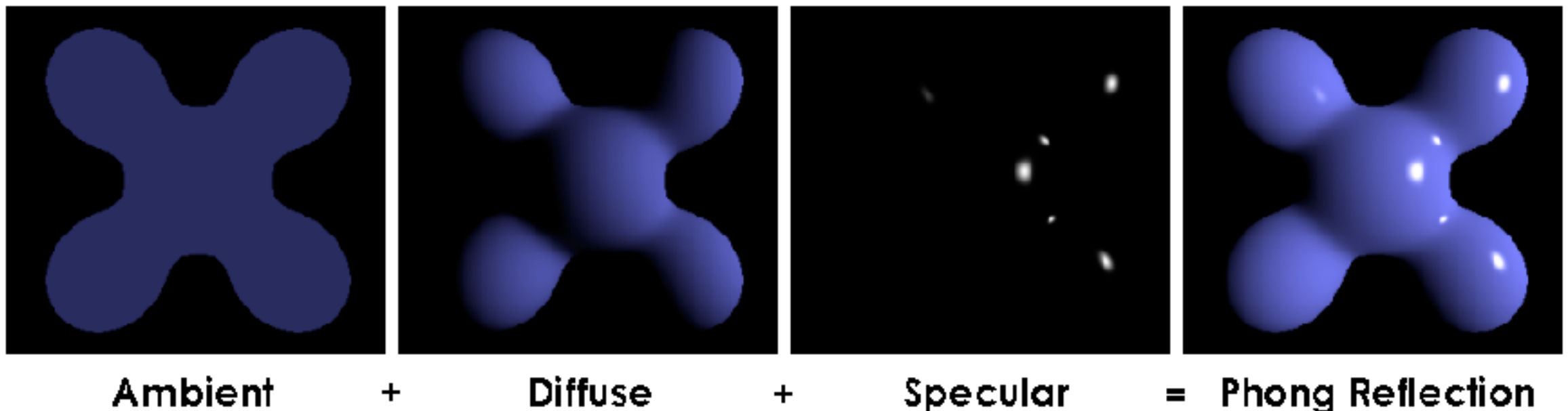
- **Global models**

- capture all illumination information
- advantages: shadows, inter-object reflection, refraction, caustics, etc.
- disadvantages: slow rendering



Phong illumination model

- *Approximate* color as a sum of the following components:
 - **ambient** (approximates the indirect illumination)
 - **diffuse** (color of the object under normal conditions)
 - **specular** (highlights on shiny objects)

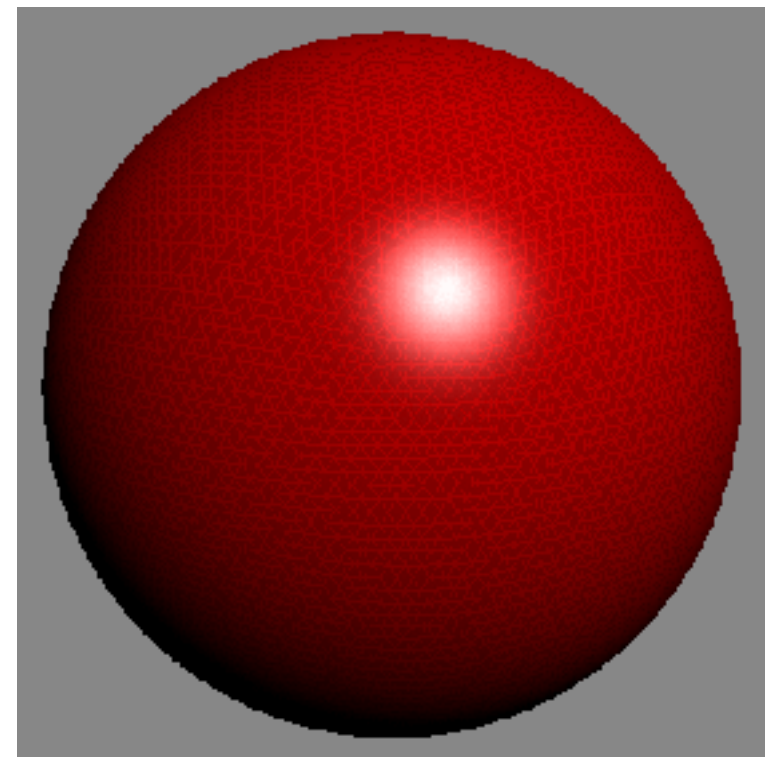
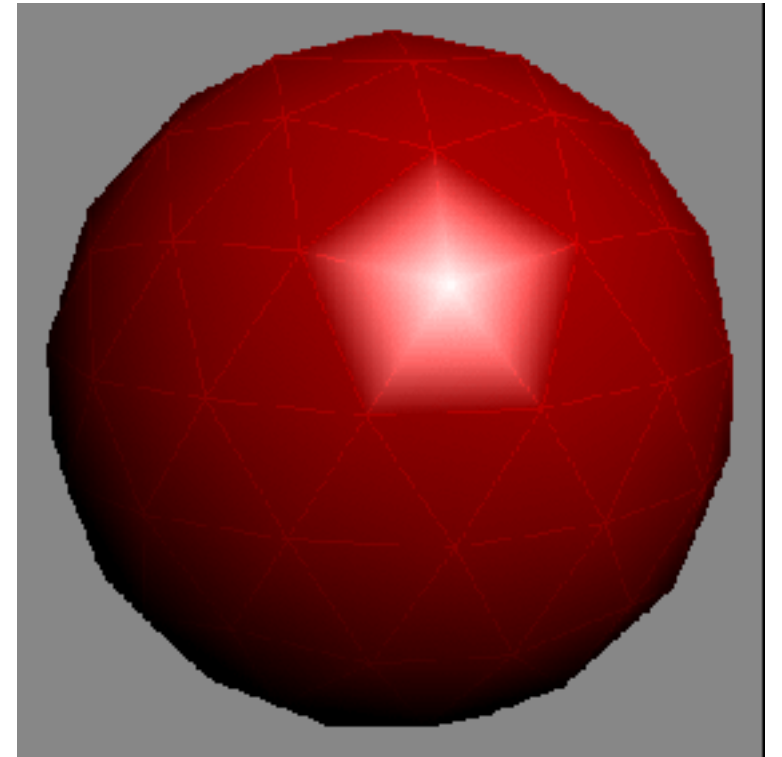


Lighting vs Shading

- **Lighting** (illumination)
 - The process of computing the intensity and color of a sample point in a scene as seen by a viewer
 - Is a function of the geometry of the scene (including the model, lights and camera and their spatial relationships) and material properties
- **Shading**
 - The process of interpolation of color at points in-between those with known lighting or illumination, typically vertices of triangles in a mesh

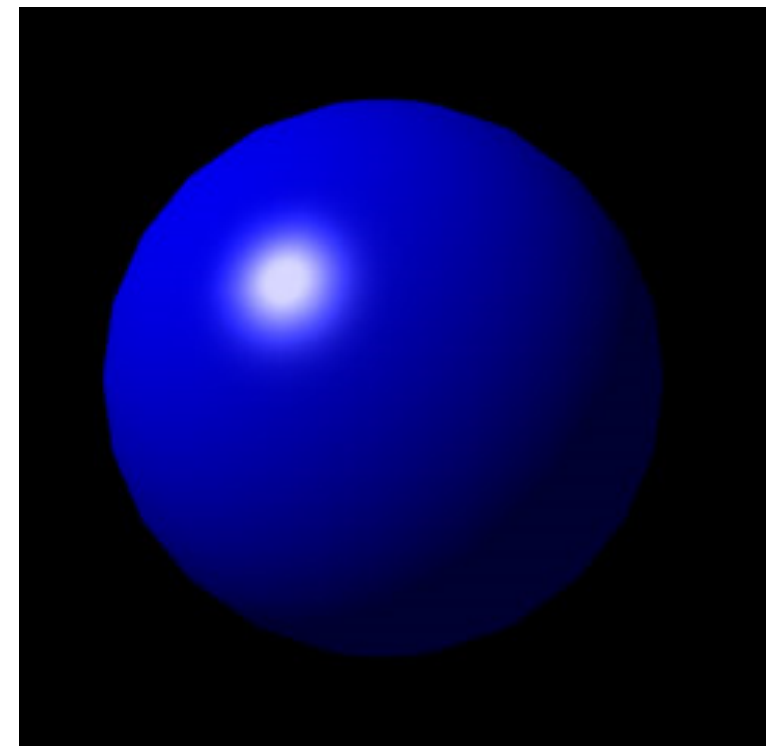
Shading - Gouraud

- Define a normal vector at each vertex
- **Lighting:** Evaluate lighting equation at each vertex using associated normal vector
- **Shading:** Each sample point's color on polygon is interpolated from color values at polygon's vertices



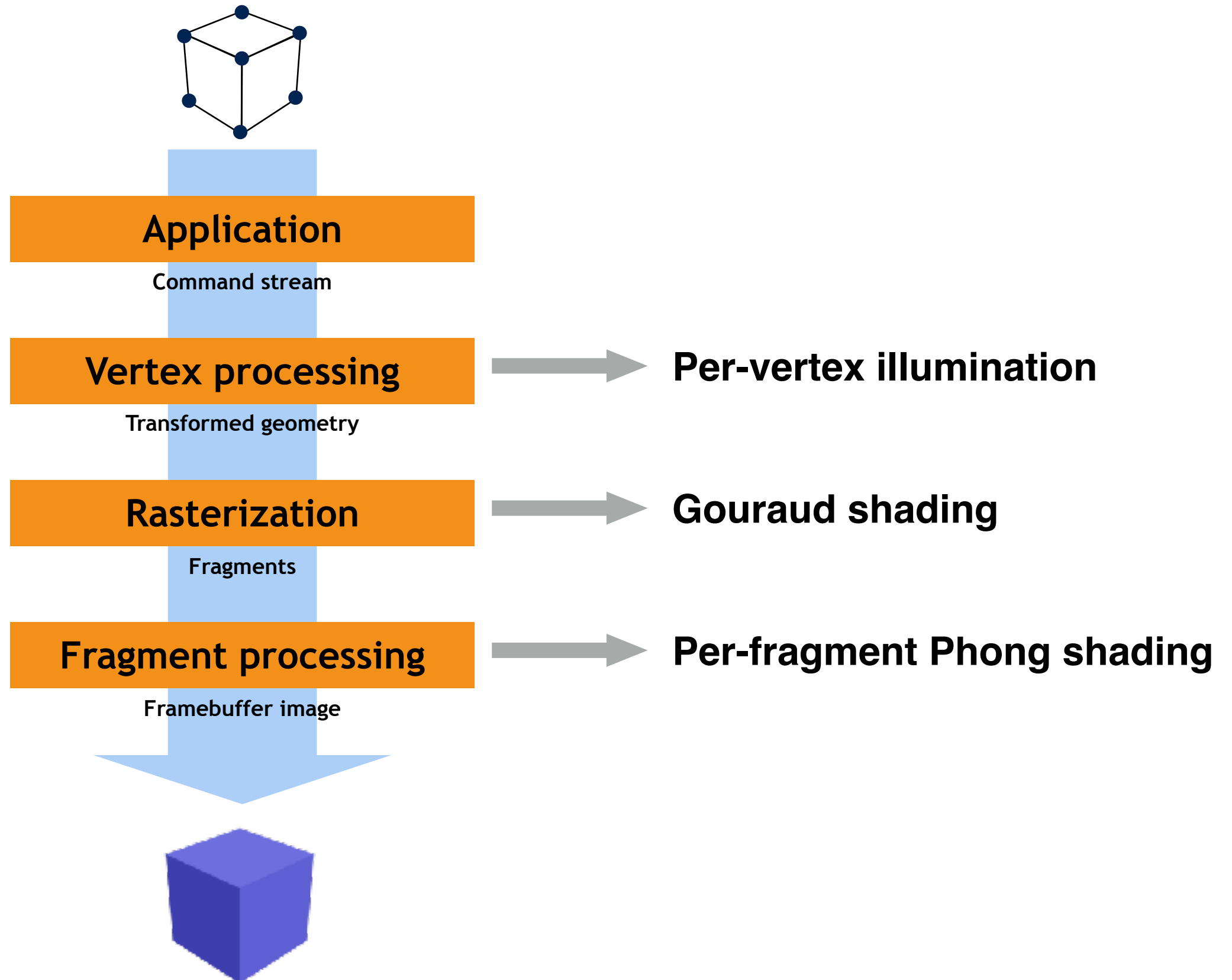
Shading - Phong

- Define a normal vector at each vertex
- **Lighting:** Evaluate lighting equation at each vertex using associated normal vector
- **Shading:** For every sample point (fragment) on polygon interpolate normals (at vertices) and compute color using lighting equation with the interpolated normal at each fragment



[Andries van Dam]

Shading models

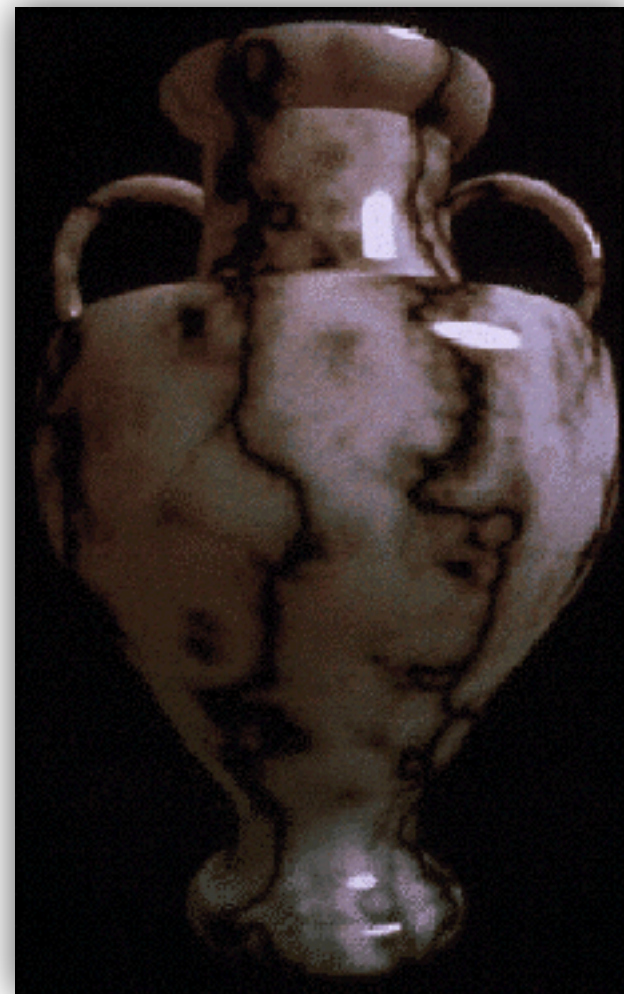


Texture mapping

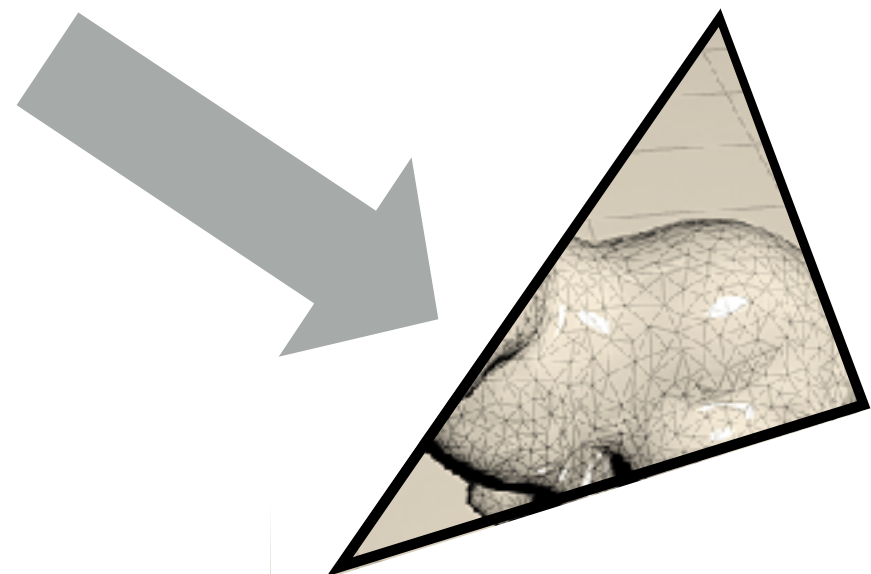
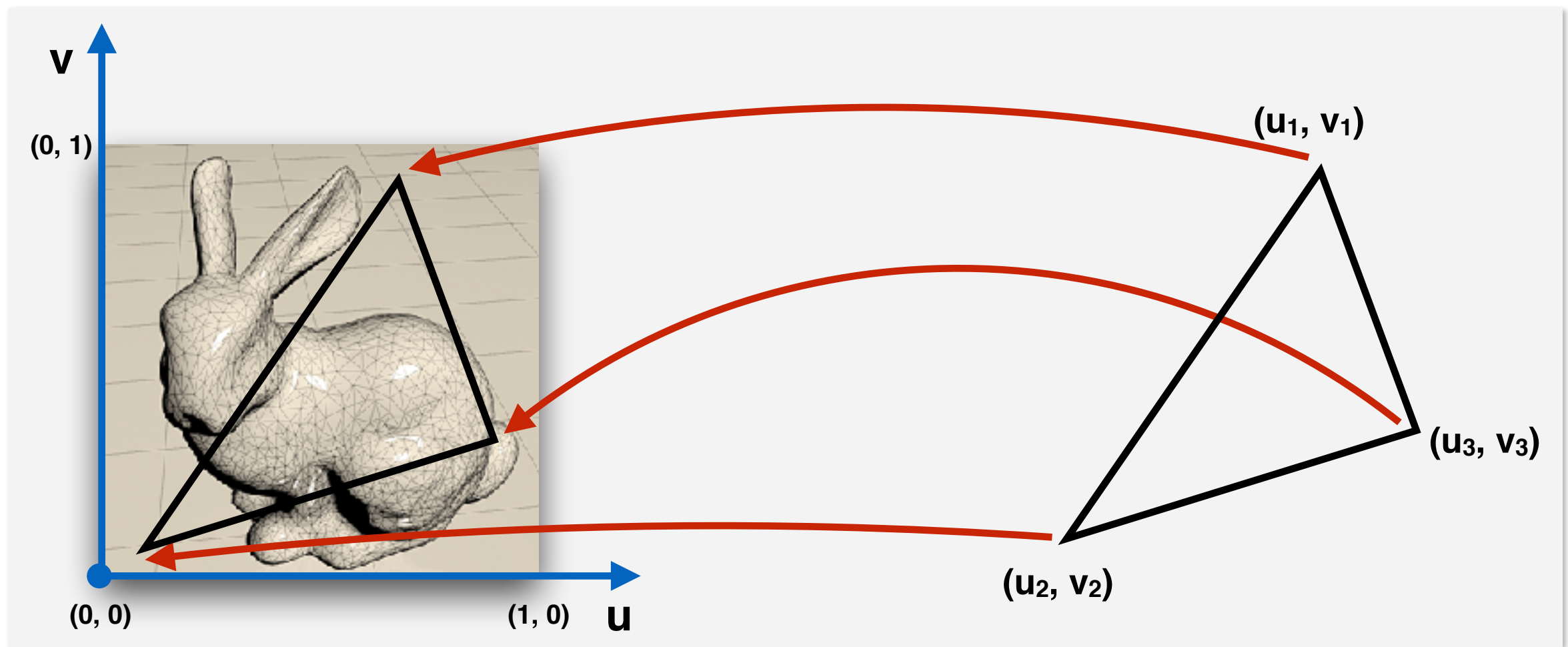
- Used to add detail to the appearance of an object (by changing reflectance)
- 2D Texture Mapping - define a mapping between a surface point and the associated texture map
- Texture coordinates
 - specified for every vertex
 - interpolated during the rasterization stage

Texture mapping

- Texture
 - 2D – graphics pattern based on matrix of pixel
 - 3D – spatial definition of the graphics pattern



2D Texture mapping



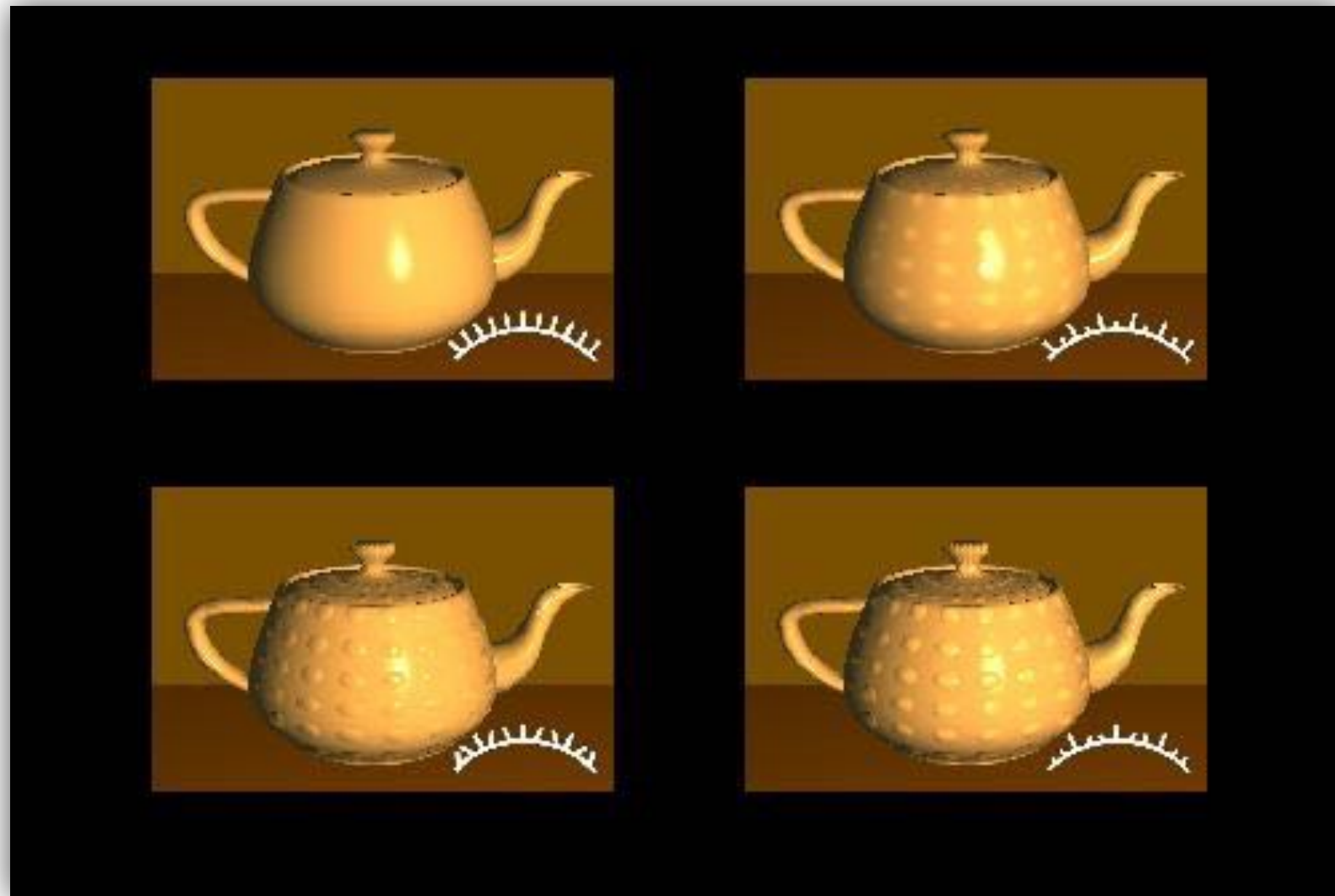
Texture mapping



[<http://andysciro.com/wp-content/uploads/2010/03/1268515673184.jpg>]

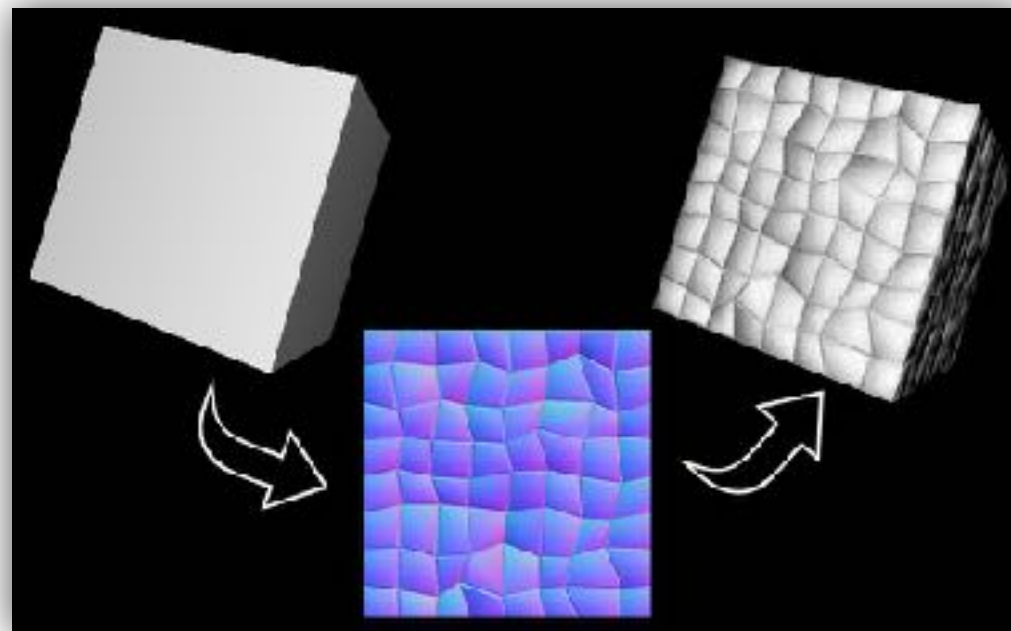
Bump mapping

- Add a bump map that perturbs the surface normal
- Does not change the geometry of the 3D models



Normal mapping

- Encodes normals as RGB values (stored as a texture)
- Replace normal at pixel with the texture value: $(R, G, B) = (x, y, z)$ of normal
- Preserve the geometry



<http://www.independentdeveloper.com/images/normalmapper.jpg>

Displacement Mapping

- A displacement map actually changes the geometry using a texture
- A common simplification is that the displacement will be in the direction of the surface normal



http://www.cggallery.com/images/tutorials/displacement_figure_06.jpg

Environment Maps

- Simulates reflections
- Uses a pre-computed texture map of the environment
- Less expensive than raytracing



<http://glasnost.itcarlow.ie/~powerk/GeneralGraphicsNotes/Theory/mappingimages/V2-cubemap.jpg>

Environment Maps



http://1.bp.blogspot.com/-224u1k6KpvM/Upa-1PhC_mI/AAAAAAAASY8/WZ2e6LPdwVg/s1600/Need-for-Speed-Rivals-18-10-2013-1.jpg

Ambient occlusion

- Rough approximation of indirect lighting
- Add realism to local reflection models (ex. Phong model)
- Represent how exposed each point in a scene is to ambient lighting

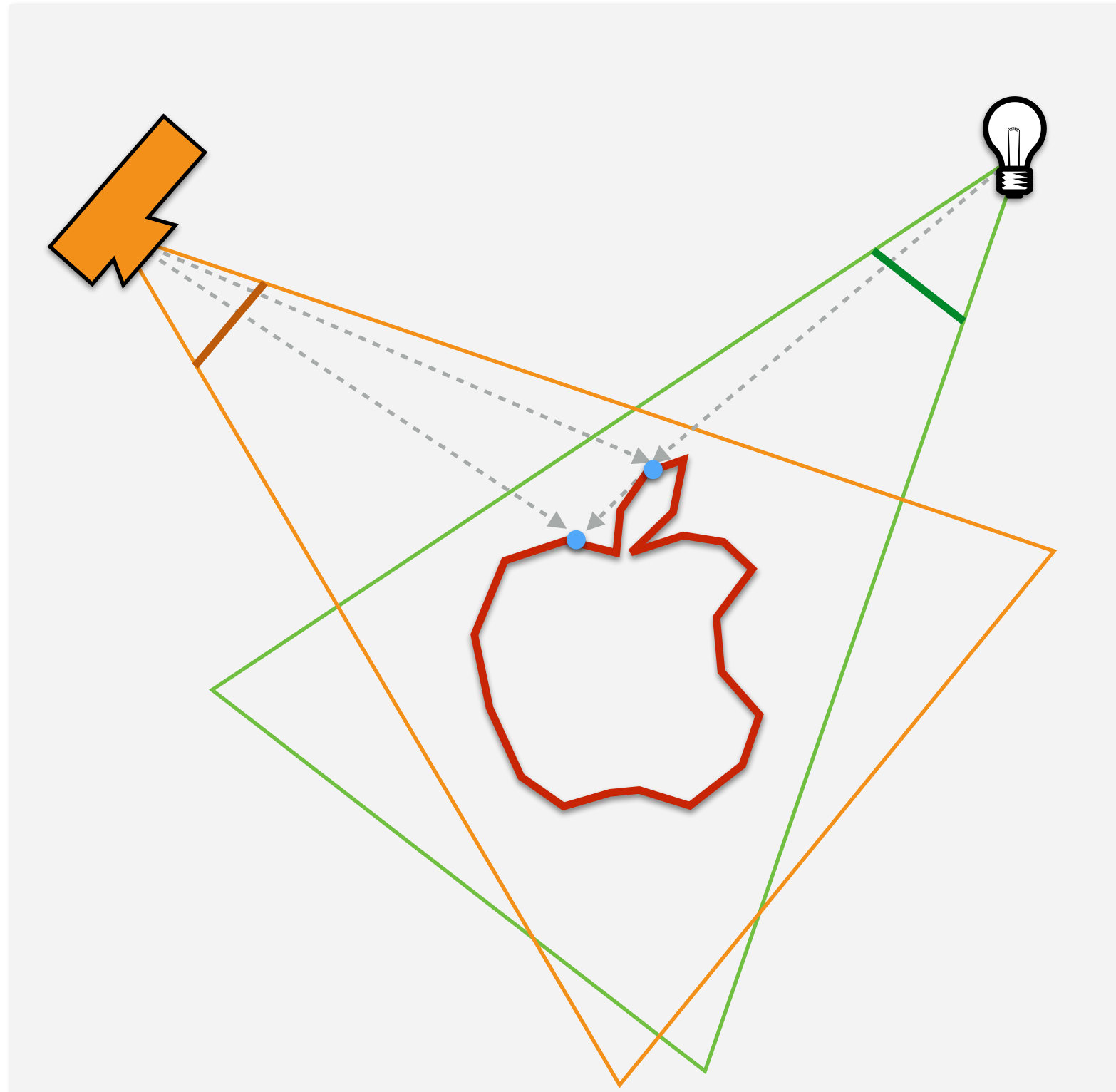


http://cgiknowledge.files.wordpress.com/2013/02/ambient_occlusin_room_image.jpeg

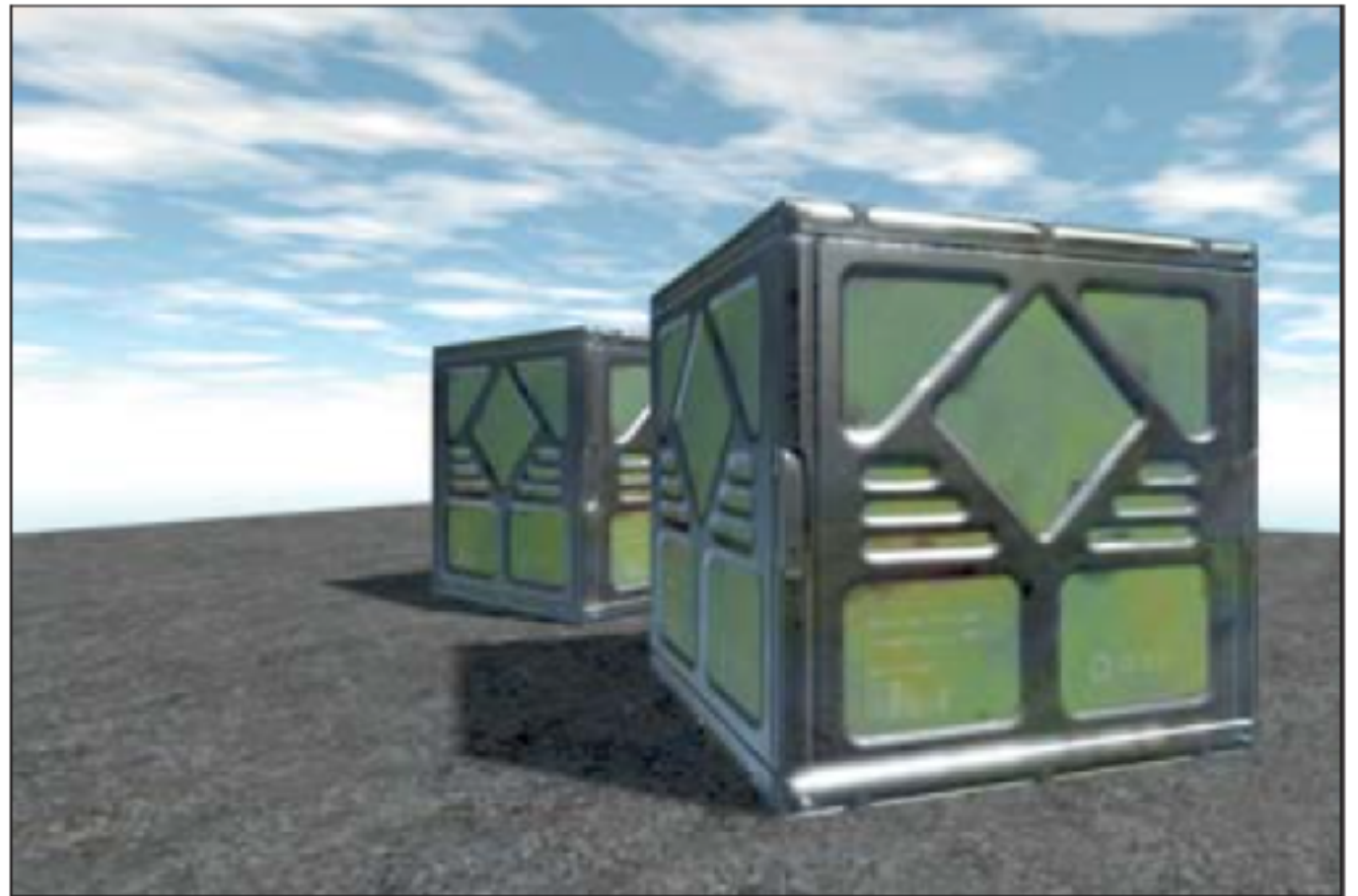
Shadow map

- Two step algorithm:
 1. **Create shadow map**
 - Render scene from light source
 - Store depth buffer
 2. **Render scene from camera**
 - Project fragment to depth buffer/light source
 - If occluder in front → dark
 - Otherwise → bright

Shadow map

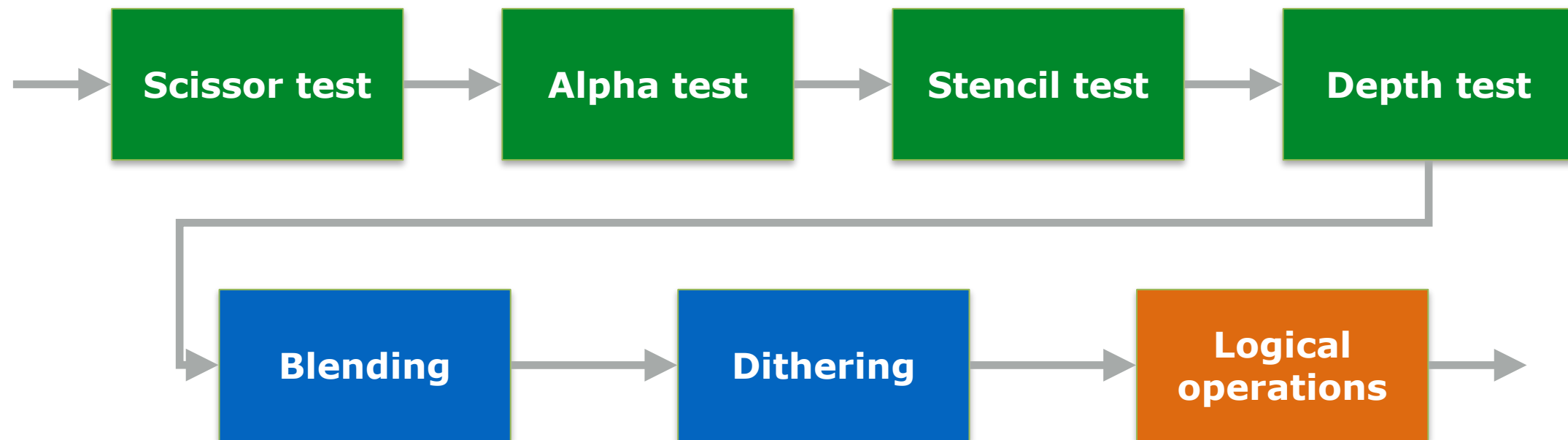


Shadow map



[Source: Computer Graphics: Principles and Practice (3rd Edition), John F. Hughes et al.]

Fragment tests



Scissor test

- Controls fragment visibility and restricts all drawing to a rectangular region
- If a fragment lies inside the rectangle, it passes the scissor test

Alpha test

- **Alpha channel**
 - additional color channel (beside the red, green and blue)
 - not a visible color
 - used for defining transparency
- **Operations**
 - compare the alpha value of an incoming fragment to a constant
 - cull all fragments that do not pass the test
 - comparison operators: $<$, $\leq$, $=$, $\geq$, $>$, $\neq$

Alpha test



Stencil test

- **Stencil Buffer**

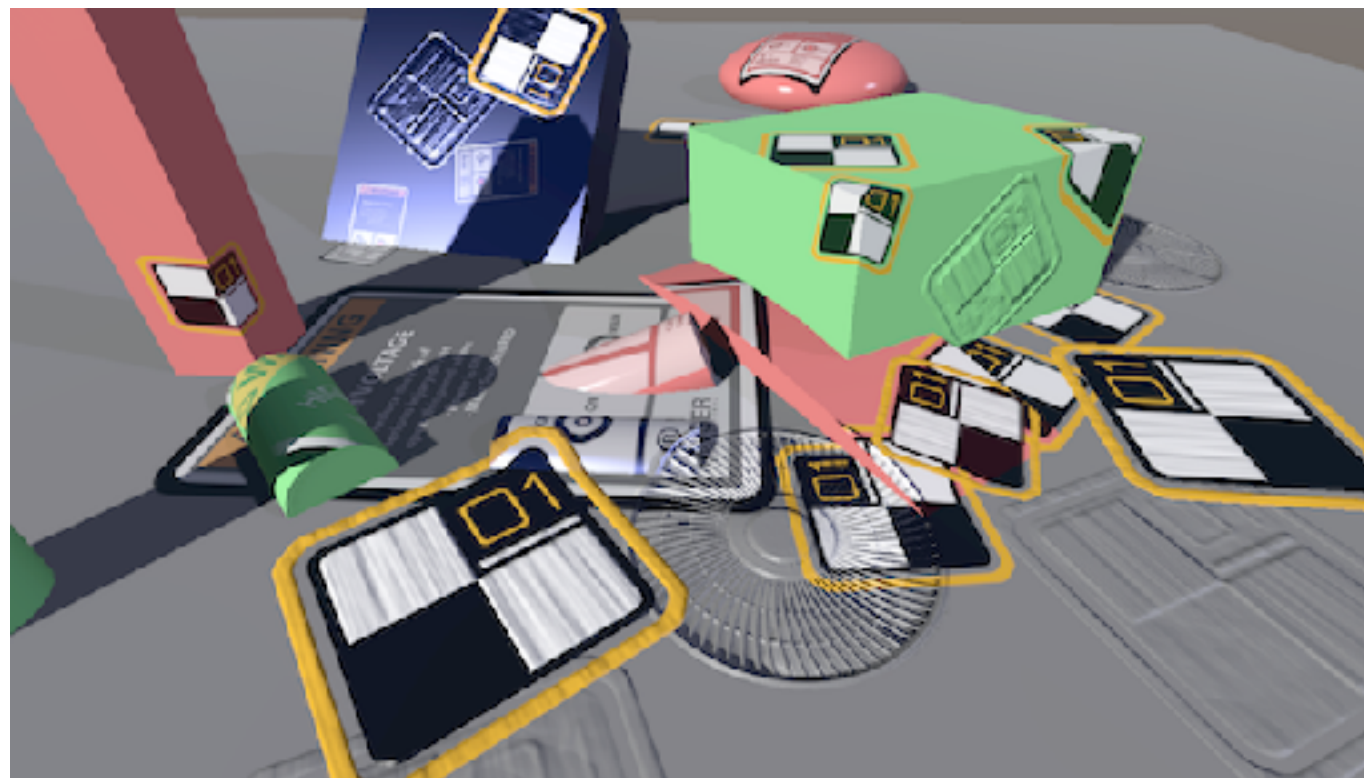
- additional buffer
- provides one channel (typically 1..8bit) for every pixel

- **Operations**

- Counting value for one pixel up or down for every fragment drawn
- Set stencil value to constant if fragment is drawn
- Clear/initialize value for all pixels
- Perform same tests as for alpha ($<$, $\leq$, ...)

Example of stencil test

- Rendering of Decals (e.g.: street demarcations)
 - Clear stencil buffer to 0
 - Draw decal polygons and set stencil to 1 for all pixels drawn
 - Draw street polygon only where stencil = 0



Depth test

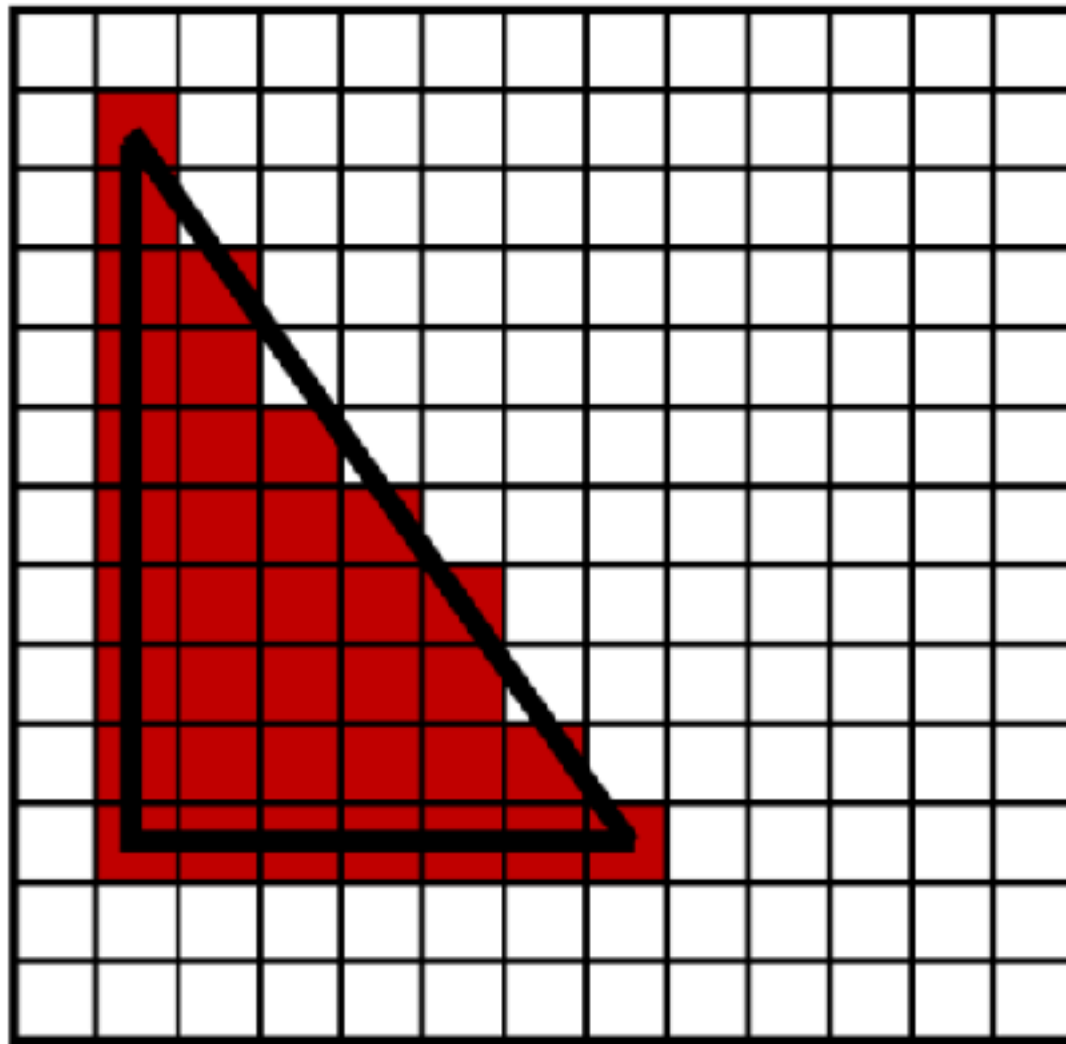
- The depth buffer keeps track of the distance between the viewpoint and the object occupying a particular pixel
- If the depth test passes, the incoming depth value replaces the value already in the depth buffer
- The depth buffer is used for hidden-surface elimination

Z-Buffer

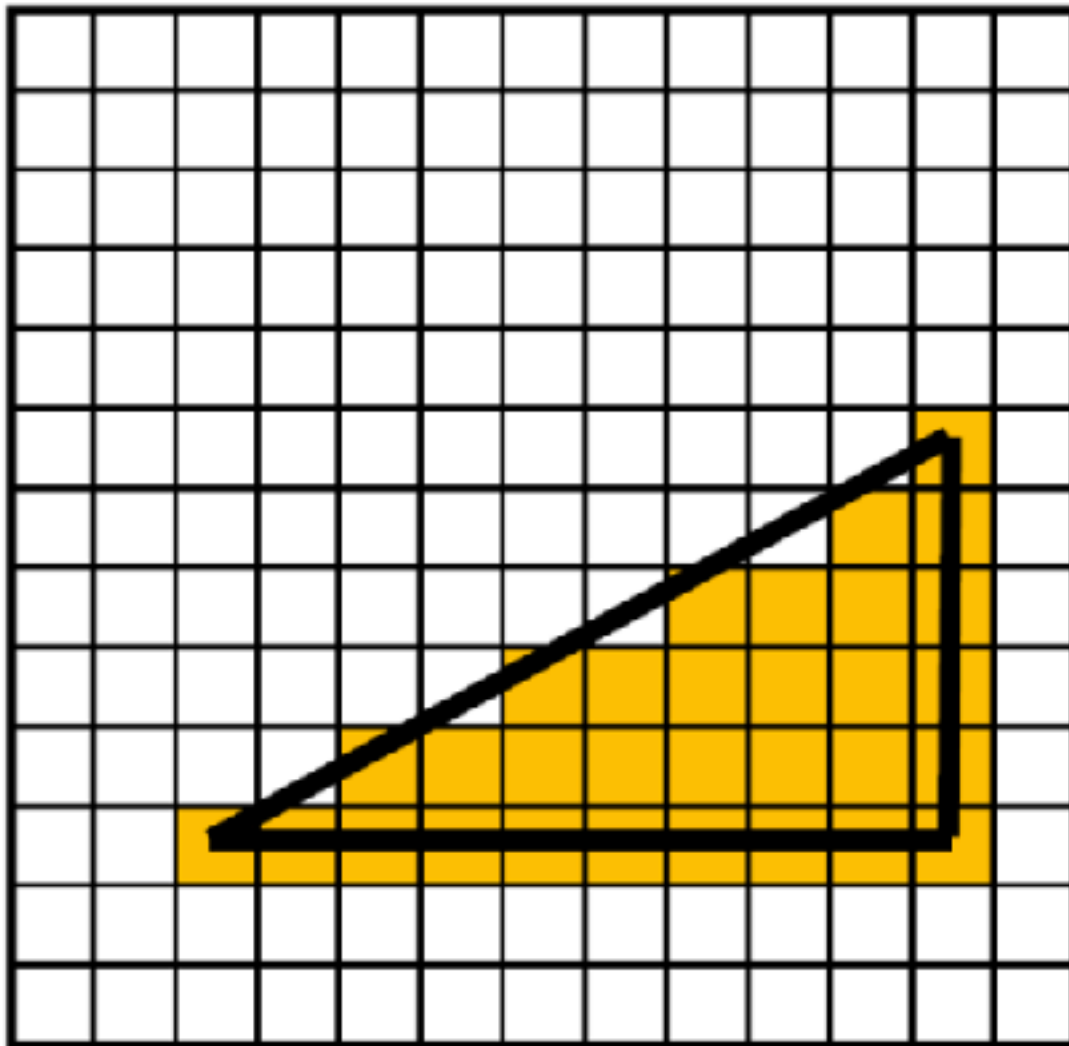
```
foreach pixel(i, j)
    zbuffer[i, j] = FAR
    color_buffer[i, j] = background_color

foreach triangle T
    foreach pixel(i, j) covered by T
        z = compute depth
        if(z > zbuffer(i, j))
            zbuffer[i, j] = z;
            color_buffer[i, j] = compute color
```

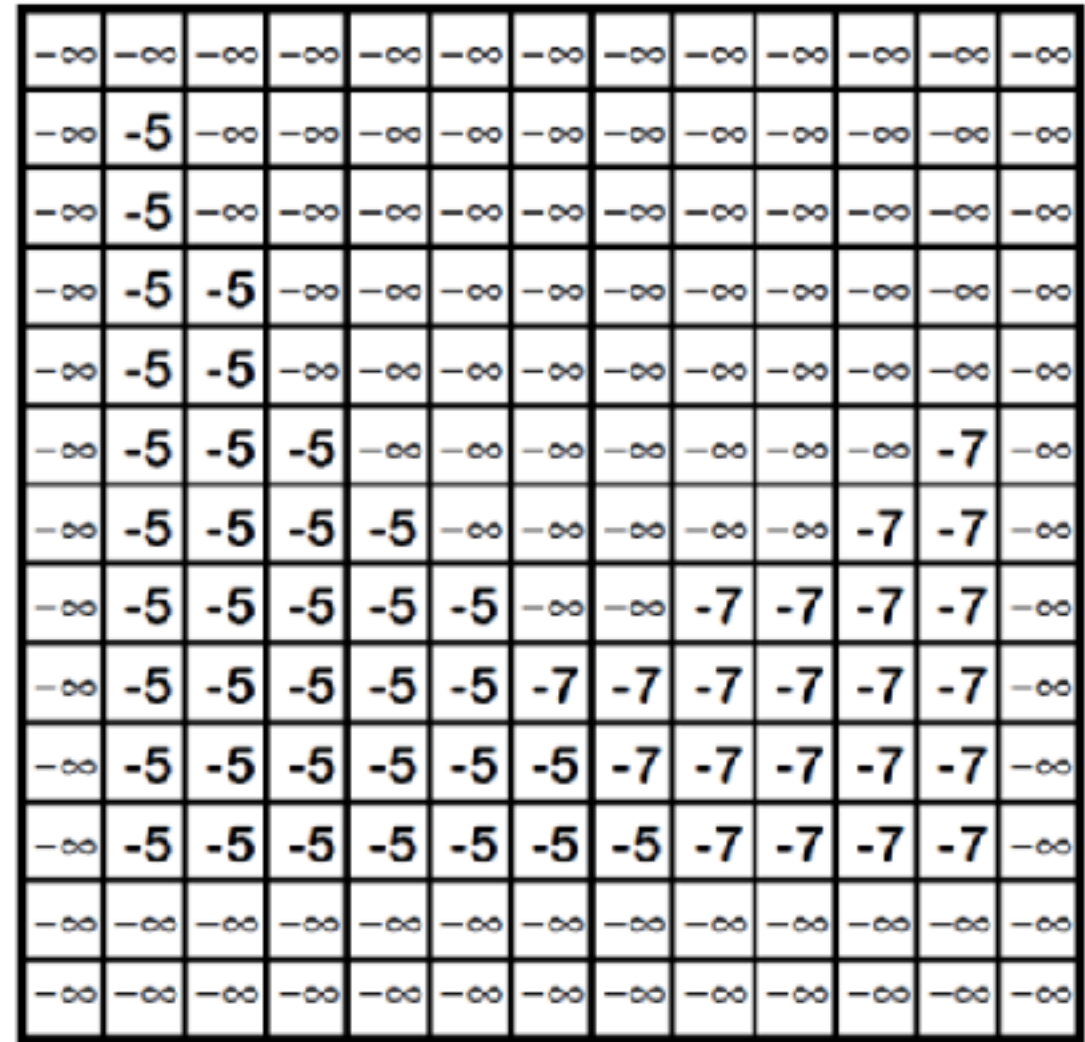
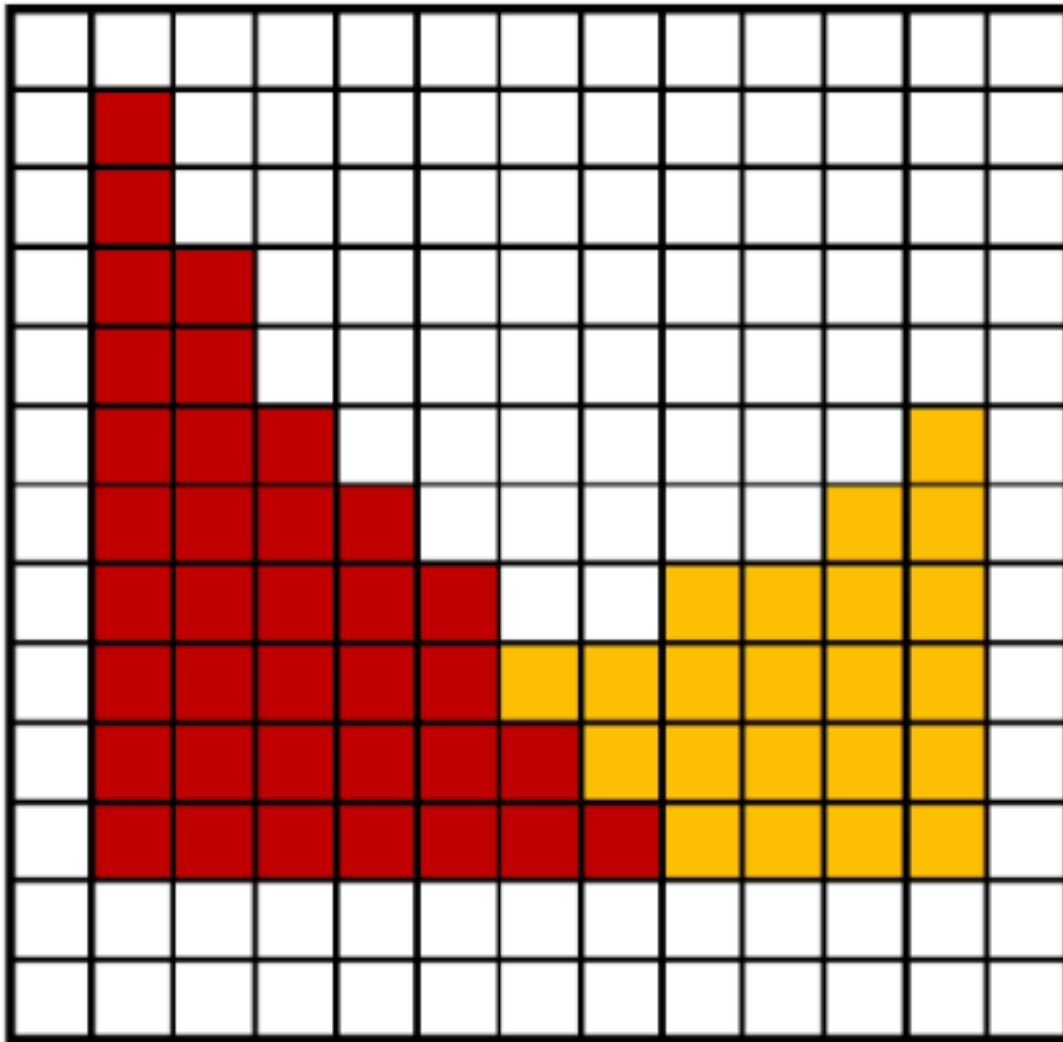
Depth test

[illegible]

Depth test

[illegible]

Depth test



Blending

- Combine the fragment color (source color) and the framebuffer color (destination color):
 - **$\text{Color} = a \cdot \text{SourceColor} + b \cdot \text{DestinationColor}$**
- Partially transparent surfaces
 - Use alpha channel (alpha= 0.1 means we see what is behind the object with 10% of the intensity, and the object itself with 90% intensity)
 - $a = 1 - \text{Source Alpha}$ and $b = \text{SourceAlpha}$

Blending

- All objects behind the transparent object must have been already drawn when the transparent object is rendered
- Draw all opaque objects first, then draw transparent objects in back-to-front order – is required a depth sort of polygons

